

Projet NFE204 FOD Semestre2 2016

Alfredo Pisco da Cruz

Les articles du journal Le Monde dans Couchbase : indexation et interrogation des données.

Introduction

Souhaitant mettre en place un moteur de recherche sémantique nous avons choisi d'utiliser un corpus de données avec un vocabulaire diversifié et riche. Le contenu du journal « Le Monde » semble idéal pour mettre en place un tel projet. Nous allons donc présenter les étapes de collecte et d'insertion des documents. Ensuite nous développons la question du stockage et de l'indexation des données avec Couchbase. Enfin nous évoquerons les possibilités offertes, une fois le système de collecte et de mise à jour en place, pour l'indexation plein texte de nos documents.

Couchbase est un système nosql en développement depuis 2011. Son histoire est liée à celle de CouchDB puisque la même personne, Damien Katz est à l'origine des 2 systèmes. Le stockage et l'indexation des données dans Couchbase reprend CouchDB toutefois le système de la gestion de la mémoire provient de Membase. Couchbase naît de la fusion de ces deux technologies. CouchDB continue toutefois à être maintenu par les équipes de Couchbase. Les deux systèmes se veulent compatibles. Couchbase est conçu pour mieux répondre aux défis de la scalabilité et de la rapidité

I Collecte des données

1.1 La source

Les abonnés du journal « Le Monde » bénéficient d'une version xml de la totalité du contenu du journal du jour. Ce fichier est contenu dans un fichier zip conjointement à la version pdf. Cette source est celle utilisée par les applications mobile et web du journal destinées à restituer les conditions de la lecture du journal papier en format numérisé.

Feuilleter le Monde depuis le site



Etant abonné au journal, la première étape du travail sera d'automatiser le téléchargement du fichier zip disponible sur le site pour les lecteurs authentifiés. Ensuite les étapes à automatiser sont, successivement :

- l'extraction du xml depuis le zip. « Dézippage »
- Le Parsing des articles
- Insertion des documents dans Couchbase

Toutes ces taches ainsi que l'ensemble du projet est réalisé en PHP. Un langage qui nous est familier.

Environnement

Pour ce projet nous utilisons un serveur unique VPS loué chez OVH avec 100 Giga de disque disponible et 8 G de RAM. Le système d'exploitation est Ubuntu 14.04.

1.2 Téléchargement et mise à jour des données.

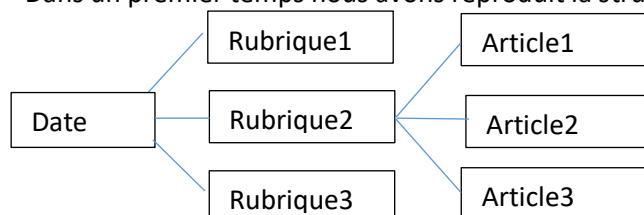
Les tâches à exécuter sont lancées par un cronjob chaque jour au milieu de l'après midi, après la mise à disposition du nouveau journal sur le site abonnes.lemonde.fr. Ceci a lieu en milieu de journée. Le rythme de la mise à jour est celui du journal papier, donc une fois par jour sauf le dimanche. Nous avons mis en place ce système depuis le mois mai 2016. Cependant nous avons pu récupérer les anciens numeros à partir du site pour les abonnés, ce qui nous permet d'avoir une base de données conséquente pour le moteur de recherche. Nous avons pu remonter jusqu'à janvier 2012.

Le Parsing

Couchbase privilégiant les documents JSON nous avons dû convertir le XML d'origine. Cette conversion implique cependant un parsing du XML puis la réécriture d'un nouveau document.

Nous n'avons conservé que la partie des données proprement dite car le xml contient également des balises liées à la mise en page, aux photos etc... Nous n'avons gardé que les champs textuels : articles, titres, dates, rubriques. A la question comment structurer le JSON différentes options se sont présentées.

Dans un premier temps nous avons reproduit la structure du XML d'origine :



Bien que techniquement ce format converti en json puisse être interrogé dans une base nosql comme Couchbase, les requêtes effectuées pour obtenir des informations élémentaires sur les articles se sont révélées assez lourdes, pour cette raison les données du Monde ont été finalement insérées dans un format de document basé sur l'article, sans imbrication. Dans la perspective de mise ne place d'un moteur de recherche sur le contenu, l'élément central est l'article et non le journal

Aperçu d'une série de documents

news	> Documents	Current page: 1	5
ID	Content		
1003695_lemonde	{ "title": "Le dimanche (en haute saison), ce n'est plus gratuit au tour..."	Edit Document	Delete
1003696_lemonde	{ "title": "Menaces sur les droits des femmes", "sub-title": "Pays conser..."	Edit Document	Delete
1003697_lemonde	{ "title": "Le grand bond de Coline Huetel", "sub-title": "La Française a..."	Edit Document	Delete
1003698_lemonde	{ "title": "De part et d'autre du sida", "sub-title": "Les traitements pe..."	Edit Document	Delete
1003699_lemonde	{ "title": "L'ONU accuse le Vatican d'avoir violé la convention des droit..."	Edit Document	Delete

L'identifiant

L'identifiant a été composé à partir de l'ID de l'article auquel nous avons ajouté le suffixe '_lemonde'.

Les champs

Les champs extraits reprennent ceux du document :

```
$id=$id."_lemonde";  
$record = array('title' => $titre, 'sub-title' => $sstitre, 'text'=>$texte, 'signature'=>$sig, 'cahier'=>$CahierNom, 'section'=>$SectionNom, 'date'=>$date);  
print x($record);
```

II Architecture et vocabulaire de Couchbase

2.1 Les Concepts

Chaque système nosql a sa propre organisation des données et son système d'interrogation plus ou moins spécifique. En allant du général vers le particulier, c'est-à-dire **le document**, qui unité de base des systèmes nosql, la première notion à envisager est celle de **cluster**.

Le cluster est une unité logique qui réunit l'ensemble des serveurs d'un système reliés entre eux. La variation du nombre des serveurs ne modifie pas cette unité. Un mécanisme de redistribution des ressources s'effectue sans changer la structure. Aucune hiérarchie n'existe entre ces machines le passage à l'échelle est ainsi automatique prise en charge par un service Couchbase au niveau du Cluster.

Le nœud correspond au niveau de la machine. Les nœuds sont égaux, il n'y a pas de topologie. Le mécanisme de **réplication** est géré au niveau des nœuds. Les données existantes sur un nœud seront répliquées sur un autre nœud, n'importe quel autre nœud. En cas de panne le réplica, ou l'un des réplicas prend la relève. La possibilité de la réplication implique au moins 2 deux. 2 réplications nécessitent 3 nœuds. Sur notre machine nous n'avons qu'un seul nœud donc pas de réplication.

Le bucket. Le bucket est le concept central de Couchbase. Il correspond à la Collection dans MongoDB. Le bucket est l'unité logique d'organisation des données. Le bucket est transversal aux nœuds. Un bucket est réparti sur l'ensemble des nœuds disponibles. Chaque bucket, donc chaque ensemble complet de données est partitionné en **vbuckets**. Chaque bucket est conçu pour recevoir 1024 vbuckets. Chacun d'eux est associé à un nœud et chaque vbucket contient un (sous) ensemble de documents. Un document est localisé par son identifiant qui est mappé à un vBucket par un hash.

Screenshot des statistiques des documents depuis l'interface de gestion de la base (Console)



Mémoire disque et mémoire RAM.

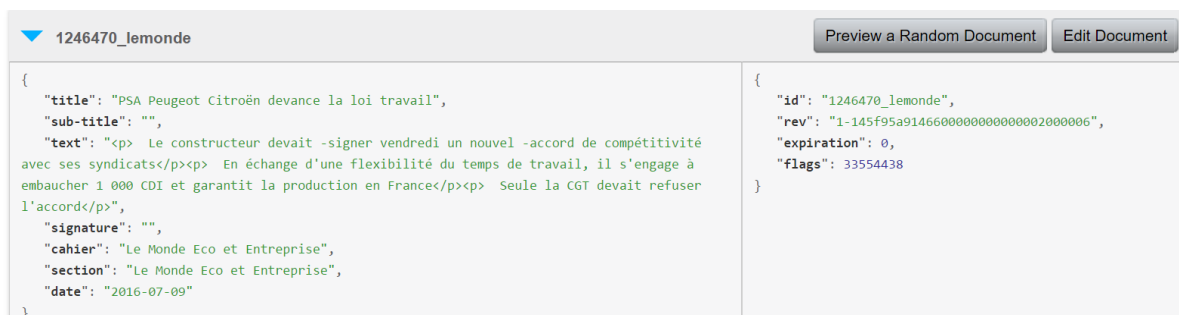
Bien entendu les documents sont stockés dans les disques toutefois Couchbase est conçu pour utiliser de manière optimale les possibilités de la mémoire RAM. Couchbase va stocker le plus de données possible afin de réduire le temps de réponse.

Les documents.

Les documents au format JSON sont privilégiés dans Couchbase. Bien que ce format ne soit pas exclusif.

Dans Couchbase chaque document ID doit être unique. Il est recommandé d'ajouter un préfixe devant le numéro de documents pour former un ID (Getting Started with Couchbase Server, MC Brown, 2011). Cette pratique peut aider l'indexation et tout simplement faire de l'identifiant une données plus lisible.

Un exemple de document du bucket news



The screenshot shows the Couchbase web interface for a document with ID `1246470_lemonde`. The document is displayed in a split view, showing the raw JSON on the left and a formatted version on the right. The JSON content is as follows:

```
{
  "title": "PSA Peugeot Citroën devance la loi travail",
  "sub-title": "",
  "text": "<p> Le constructeur devait -signer vendredi un nouvel -accord de compétitivité avec ses syndicats</p><p> En échange d'une flexibilité du temps de travail, il s'engage à embaucher 1 000 CDI et garantit la production en France</p><p> Seule la CGT devait refuser l'accord</p>",
  "signature": "",
  "cahier": "Le Monde Eco et Entreprise",
  "section": "Le Monde Eco et Entreprise",
  "date": "2016-07-09"
}
```

The right pane shows the formatted JSON with the following fields:

```
{
  "id": "1246470_lemonde",
  "rev": "1-145f95a914660000000000002000006",
  "expiration": 0,
  "flags": 33554438
}
```

Buttons at the top right include "Preview a Random Document" and "Edit Document".

Après avoir présenté l'architecture de Couchbase nous pouvons décrire les opérations effectués sur les données stockées.

III Interroger les documents

3.1 Interrogation par ID

Pour les requêtes les plus élémentaires sont basées sur l'ID. Elles concernant la création, l'interrogation, la mise à jour et la suppression dans la base de documents (CRUD opérations), Couchbase propose les fonctions suivantes :

- `insert(docid,document)`
- `get(docid)`
- `replace(docid,document)`
- `remove(docid)`


```

130 //insert in couchbase
131 function InsertCB($id,$document){
132 $myCluster = new CouchbaseCluster();
133 $myBucket = $myCluster->openBucket("news");
134
135 $myBucket->upsert($id, $document);
136 }
137 }
138 //Get document
139 function GetDoc($docname){
140 $myCluster = new CouchbaseCluster();
141 $myBucket = $myCluster->openBucket("news");
142 $res = $myBucket->get($docname);
143 }
144 }
145 //delete
146 function RemoveDoc($docname){
147 $myCluster = new CouchbaseCluster();
148 $myBucket = $myCluster->openBucket();
149 $myBucket->remove($docname);
150 }
151 }

```

Couchbase, contrairement à CouchDB, ne nous propose pas d'API REST pour effectuer ces opérations avec les fonctions PUT,DELETE, GET ou POST. Toutefois il sera possible d'envoyer des requêtes N1QL , équivalentes (SELECT,DELETE,UPDATE) par http comme nous le verrons plus tard.

3.2 Interrogation de la base sans le documentID :

Les deux moyens proposés par Couchbase pour ramener des documents sans connaître préalablement leur ID sont :

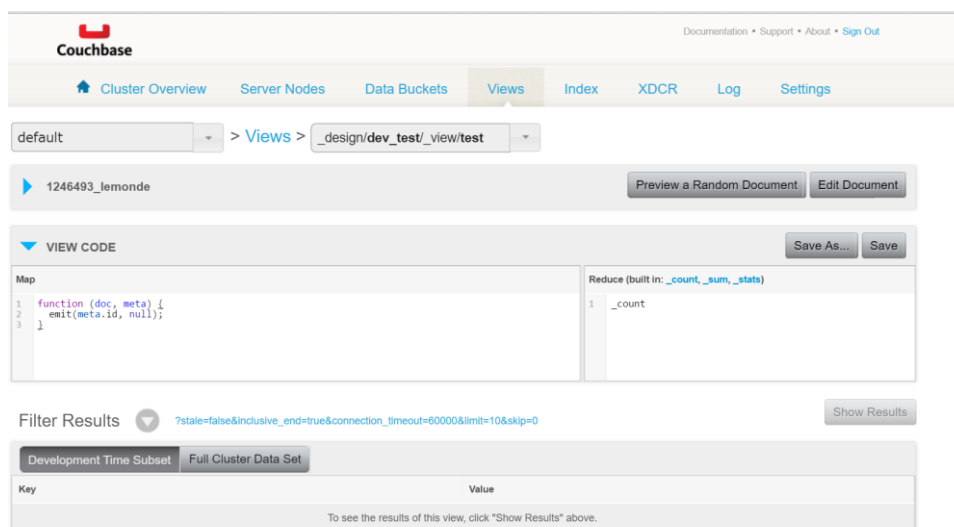
- Les views Map Reduce en Javascript. Il s'agit de l'outil natif de Couchbase. C'est aussi l'instrument d'interrogation des documents le plus répandu parmi les bases noSql.
- Les Requêtes N1QL. ("*nickel*") , il s'agit du DSL de Couchbase disponible depuis 2015. Tout d'abord il s'agissait d'un module distinct optionnel. Désormais cet outil est intégré dans le cœur du programme depuis la version 4.0. Ce langage est une adaptation de la syntaxe usuelle du SQL prévue ici pour une interrogation des documents JSON, avec plusieurs niveaux d'imbrication.

3.2.1 Les views Map Reduce.

3.2.1.1 Par la « console »

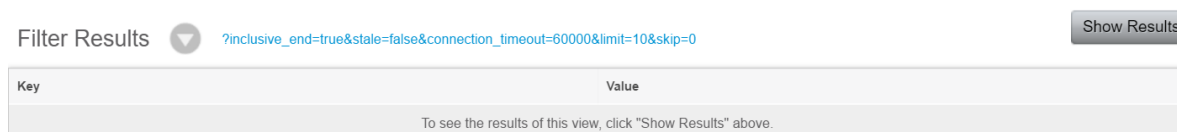
Elles vont créer des index basés sur des champs et des critères de filtrage. Les index vont permettre l'accès rapide aux documents stockés sur le disque.

L'interface web, la « console », contient un éditeur de views. A partir d'un code de base il est possible d'écrire une fonction map et une fonction de reduce. Couchbase simplifie même cette dernière étape avec des « built in functions » : `_count`, `_sum` et `_stats`. Les agrégations peuvent ainsi être réalisées rapidement à partir des éléments clé-valeur émis par la fonction de map.

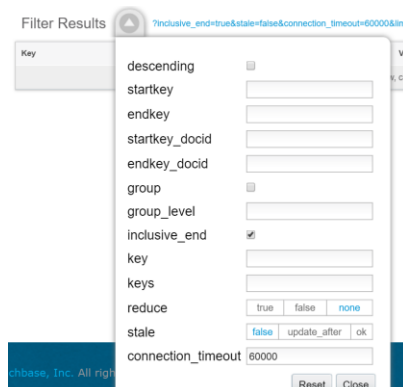


L'interface web permet un basculement des views entre le mode développement et le mode production. Le mode développement permettant de tester une nouvelle view sur un seul vBucket, c'est-à-dire une seule partition du jeu de données. Une fois satisfait de notre fonction elle est enregistrée et l'indexation de l'ensemble des documents peut s'effectuer.

Le résultat des views peut être visualisé en http. Une url est générée pour chaque view :

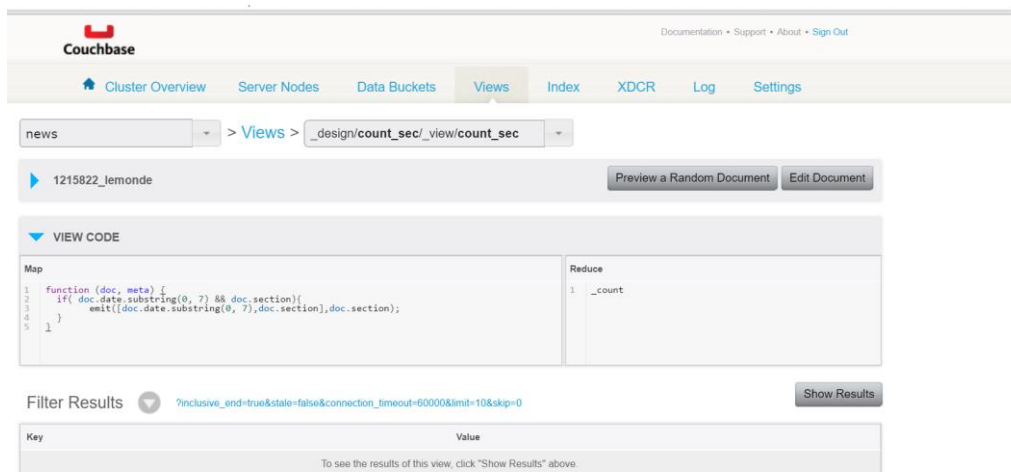


Cette url contient notamment des paramètres d'affichage (limit), de cohérence (stale) et aussi de niveau d'agrégation (group level). Le paramètre group est particulièrement utile lorsque l'on utilise les fonctions `_count`, `_sum` avec de multiples clés. Ce paramétrage s'effectue par le bouton « Filter results » :



L'utilisation des built-in fonctions retourne rapidement le résultat d'agrégations avancées. Voici un exemple d'utilisation de cet outil sur notre jeu de données « Le Monde » :

Nous voulons effectuer un comptage du nombre d'articles par Rubrique (section) et par mois. En vue de réaliser, par exemple, une visualisation de l'évolution du poids des différentes rubriques au cours des dernières années :



Résultat disponible en ligne :

http://vps82423.ovh.net:8092/news/_design/count_sec_view/count_sec?group_level=2&inclusive_end=true&stale=false&connection_timeout=60000&limit=10&skip=0.

3.2.1.2.1 En PHP :

Dans la pratique la view est interrogée par un script par des fonctions prévues. Ainsi en PHP nous interrogeons la même view par ce code :

```
$myCluster = new CouchbaseCluster();
$myBucket = $myCluster->openBucket('news');
$query = CouchbaseViewQuery::from('count_sec', 'count_sec')->group_level(2)->limit(10)->order(CouchbaseViewQuery::ORDER_DESCENDING);

$res = $myBucket->query($query);
var_dump($res);
```

La fonction CouchbaseViewQuery() prend les mêmes paramètres (+ ou moins) que ceux proposés par le menu Filter Results dans la Console.

Mongodb présentait un avantage sur Couchbase lors des premières versions : son langage d'interrogation sur les documents JSON. Celui-ci étant plus simple que l'écriture de View Map Reduce en JavaScript.

A partir de la version 4.0 de Couchbase, le langage N1QL devient le moyen privilégié pour interroger la base. Celui-ci est présenté comme un moyen pratique, puisque le SQL est un langage simple et performant et potentiellement très rapide.

3.2.2 Ecrire des queries N1QL

Comparativement aux Views, nous avons davantage d'outils pour effectuer des requêtes N1QL aussi bien dans un contexte de test que dans le cadre d'une application.

cqb :

En ligne de commande il existe un utilitaire nommé **cqb**, également accessible par le shell principal cbc.

Exemple de query par cqb

```
root@vps82423:~# cd /opt/couchbase/bin/
root@vps82423:/opt/couchbase/bin# ./cbq
Couchbase query shell connected to http://localhost:8093/. Type Ctrl-D to exit.
cbq> SELECT DISTINCT n.section,COUNT(*),SUBSTR(n.date,0,7) FROM 'news' n GROUP BY n.section,SUBSTR(n.date,0,7) ORDER BY SUBSTR(n.date,0,7) desc
```

Query Workbench :

Il existe une interface web de cqb appelée **Query Workbench**. Le confort pour écrire les requêtes est bien supérieur et la visualisation des résultats est possible au format tableau, json ou arborescence.

Exemple de query avec Query Workbench

API REST :

Les requêtes N1QL peuvent être lancées en REST, ce qui n'était pas possible pour les views.

Requête REST avec curl

```
* Connection #0 to host localhost left intact
root@vps82423:/opt/couchbase/bin# curl -v http://localhost:8093/query/service -d\
> '{"statement":"SELECT DISTINCT n.section,COUNT(*),SUBSTR(n.date,0,7) FROM 'news' n GROUP BY n.section,SUBSTR(n.date,0,7) ORDER BY SUBSTR(n.date,0,7) desc"}'
```

SDK :

En version script pour les développeurs d'applications le SDK Couchbase propose des fonctions spécifiques dans le même style que pour interroger les Views.

En PHP nous écrivons :

Query N1QL en PHP

```
//Working with N1QL queries

$myCluster = new CouchbaseCluster();
$myBucket = $myCluster->openBucket();
$myBucket->enableN1ql(array('http://1.1.1.1:8093/', 'http://1.1.1.2:8093/'));

$myn1ql=<<<myn1ql
SELECT DISTINCT n.section,COUNT(*),SUBSTR(n.date,0,7) FROM 'news' n
GROUP BY n.section,SUBSTR(n.date,0,7)
ORDER BY SUBSTR(n.date,0,7) desc
myn1ql;
$query = CouchbaseN1qlQuery::fromString($myn1ql);
$res = $myBucket->query($query);
var_dump($res);
/*
```

Les fonctions **enableN1ql()** et **CouchbaseN1qlQuery()** vont permettre lancer notre query dans un script PHP.

3.2.3 Les performances entre NQL1 et Views.

La documentation de Couchbase met en avant les performances supérieures des Query NQL1 par rapport aux Views. Cet avantage se vérifie si les index secondaires (Global Secondary Index) sont correctement utilisés. Sans index la requête prendra plus de temps. Couchbase permet en effet d'effectuer des queries N1QL sans indexation (ad hoc), ce qui n'est pas possible avec les views. Toutefois dans ce cas, les données seront ici interrogées séquentiellement document par document ce qui augmente le temps de recherche.

Nous allons créer différents index sur notre bucket news et constater la performance retournée par le Query Workbench.

Une fois créé l'index primaire :

```
Definition: CREATE PRIMARY INDEX `#primary` ON `news` USING GSI
```

Lançons la requête :

N1QL History 3/8 Clear History Result limit: 51000 Execute

```
1 SELECT DISTINCT n.section, COUNT(*), SUBSTR(n.date, 0, 7) FROM `news` n GROUP BY n.section, SUBSTR(n.date, 0, 7) ORDER BY SUBSTR(n.date, 0, 7) desc
```

Results JSON Table Tree Save

Status: success Elapsed: 36.01s Execution: 36.01s Result Count: 1539 Result Size: 159630B

\$1	\$2	section
2016-06	2016-06	Télévision

Le temps de traitement est de 36s.

Créons un index secondaire sur le champ date (par mois) et le champ section :

```
Definition: CREATE INDEX `idxcountsec` ON `news` (substr(`date`, 0, 7), `section`) USING GSI
```

En relançant simplement la requête le temps de traitement est identique.

C'est en ajoutant dans la clause WHERE , NOT MISSING que nous parvenons véritablement à optimiser la requête :

Avec **IS NOT MISSING**

N1QL History 10/10 Clear History Result limit: 51000 Execute

```
1 SELECT DISTINCT n.section, COUNT(*), SUBSTR(n.date, 0, 7) FROM `news` n
2 WHERE n.section IS NOT MISSING
3 AND SUBSTR(n.date, 0, 7) IS NOT MISSING
4 GROUP BY n.section, SUBSTR(n.date, 0, 7) ORDER BY SUBSTR(n.date, 0, 7) desc
```

Results JSON Table Tree Save

Status: success Elapsed: 5.12s Execution: 5.12s Result Count: 1538 Result Size: 159570B

\$1	\$2	section
2016-06	2016-06	Télévision

Le temps de traitement est désormais de 5 s !

La fonction EXPLAIN appliquée à chacune de ces requêtes permet de voir que dans le premier cas (sans NOT MISSING) la requête a interrogé l'index primaire. C'est dans la deuxième que l'index GSI idxcountsec a été interrogé

Donc plus la requête coïncide avec l'index plus le résultat est performant. Par ailleurs dans le cas présent les données que nous recherchons sont déjà contenues dans l'index. C'est-à-dire la liste des mois, la section et le nombre d'articles pour chaque mois et chaque section. Le service de requête n'a pas besoin ramener les données depuis leur emplacement. Il s'agit d'une COVERING QUERY.

La performance des views :

En lançant la fonction Map Reduce depuis le lien proposé dans la Console le temps d'exécution est également de 5 secondes environ :

```

← → ↺ ⌂ vps82423.ovh.net:8092/news/_design/count_sec/_view/count_sec?group_level=2&inclusive_end=true&stale=false&connection_timeout=60000&limit=5000&skip=0

{"rows": [
  {"key": ["2012-01", "& vous"], "value": 110},
  {"key": ["2012-01", "Carnet"], "value": 18},
  {"key": ["2012-01", "Culture"], "value": 223},
  {"key": ["2012-01", "D&#233; bats"], "value": 96},
  {"key": ["2012-01", "Derni&#232;re Heure"], "value": 62},
  {"key": ["2012-01", "Economie - Entreprises"], "value": 422},
  {"key": ["2012-01", "Editorial - Analyses"], "value": 93},
  {"key": ["2012-01", "En ligne"], "value": 72},
  {"key": ["2012-01", "Enquete"], "value": 22},
  {"key": ["2012-01", "Europe"], "value": 19},
  {"key": ["2012-01", "International"], "value": 242},
  {"key": ["2012-01", "L'Oeil du Monde"], "value": 4},
  {"key": ["2012-01", "Le bal des festivals"], "value": 358},
  {"key": ["2012-01", "Le Monde Culture et id&#233;es"], "value": 45},
  {"key": ["2012-01", "Le Monde des Livres"], "value": 140},
  {"key": ["2012-01", "Le Monde Economie(m&#223;e)"], "value": 50},
  {"key": ["2012-01", "Le Monde Education"], "value": 23},
  {"key": ["2012-01", "Le Monde G&#223;o et politique"], "value": 46},

```

En activant le paramètre `stale` à `FALSE` nous sollicitons un rafraîchissement de l'index avant le renvoi des résultats. Sans ce rafraîchissement (`stale=OK`) l'affichage des résultats est immédiat. La rapidité de mise à jour des index est liée au fait que les changements ne se font qu'en cas de changement dans les données.

Ces performances des `views map reduce` semblent relativiser l'intérêt de `N1QL`.

En réalité c'est à une plus grande échelle sur plusieurs nœuds que son avantage doit apparaître. À la différence des `query map reduce`, les index gérés ne sont pas stockés localement, ils ne sont pas associés aux données. Le temps pour le regroupement et le transfert des résultats depuis chaque `vBucket` va augmenter à la mesure de la quantité de données présentes sur les différents nœuds. Les `GSI` ne procèdent pas comme les `Views` par « scatter-gather ». Chaque index est centralisé sur l'un des nœuds et leur mise à jour des index se fait indépendamment de la mise à jour des données.

Jointures :

Cette architecture où les index sont gérés indépendamment des données permet de retrouver l'un des avantages propres des bases relationnelles : les jointures. Dans Couchbase il est ainsi possible et facile d'effectuer des requêtes sur plusieurs buckets.

Nous allons voir désormais comment les données et les différents types d'index sont organisés dans les serveurs.

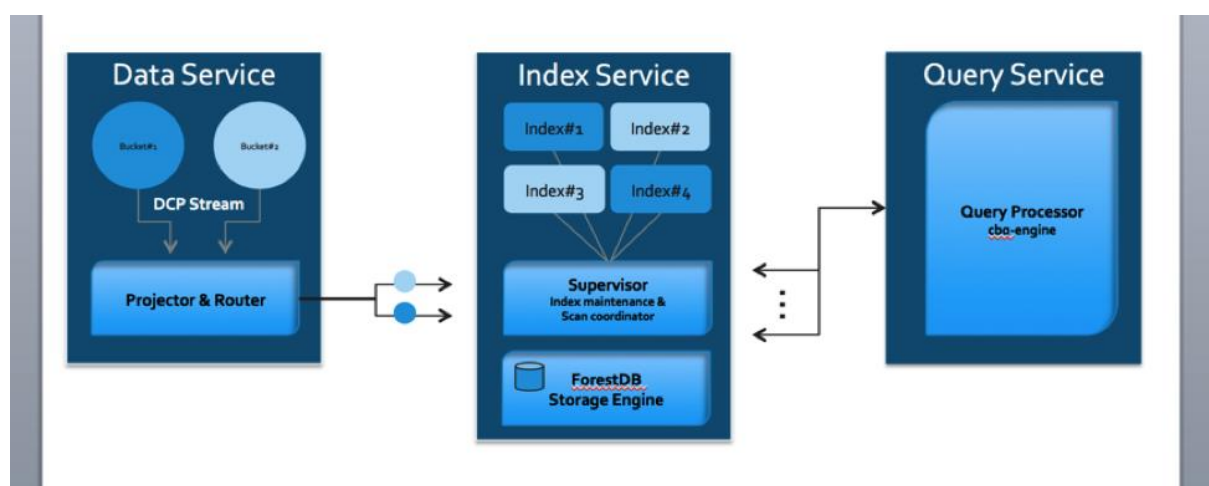
IV Les index dans l'architecture de Couchbase :

Le système de requêtes N1QL est basé sur une indexation particulière : les GSI. Global Secondary Indexes qui sont centralisés puis distribués sur les différents nœuds.

Les requêtes N1QL interrogent donc d'emblée l'ensemble des documents (où du moins leur méta données) sur tous les nœuds.

Ce système conserve donc l'avantage des views map reduce : la possibilité de passage à l'échelle. En effet l'ajout de nœuds dans le Cluster s'accompagnera automatiquement d'une redistribution des index secondaires sur tous les nœuds y compris les nouveaux.

Les index GSI pour les requêtes N1QL sont conçus de la même manière que les index B tree des bases relationnelles mais avec des possibilités d'échange des informations entre les différents nœuds : le graphique suivant provenant de la documentation de Couchbase illustre la circulation des informations entre les données, les index et la requête :



Le superviseur, au niveau de l'un des nœuds achemine les informations vers les informations en provenance des buckets et en provenance de la requête vers l'index unique localisé sur l'un des nœuds. Le moteur ForestDB est en fait une amélioration conçue pour les données massives de l'index B tree. Le ForestDB privilégie l'augmentation du nombre de clés plutôt que l'augmentation des niveaux de profondeur qui peuvent s'avérer coûteux à très grande échelle.

V Couchbase et l'indexation Full Text

Nous avons présenté les possibilités d'indexation et de recherche offertes par Couchbase sur la version 4.0. La recherche textuelle étant besoin courant dans le domaine des bases de données documentaires, la dernière version en date de Couchbase 4.5 (payante) incorpore un moteur de recherche full text : bleve. Auparavant cet outil devait être ajouté comme plugin. Avant de le tester il convient de présenter ce qui reste toutefois la meilleure option pour ajouter un moteur de recherche à Couchbase : Elasticsearch.

5.1 Couchbase et Elasticsearch

Couchbase offre une possibilité particulièrement efficace d'utiliser la puissance d'Elasticsearch et la finesse de ses indexs et queries.

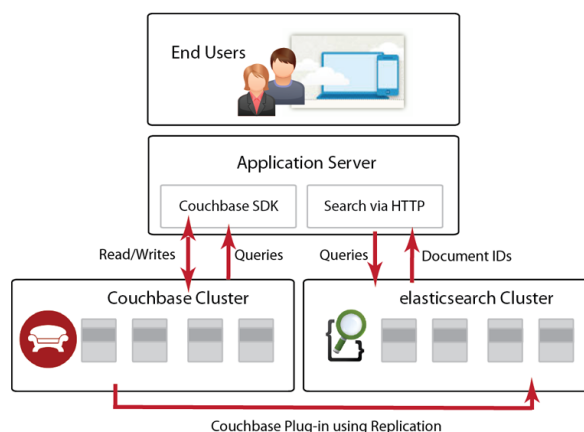
L'onglet XDCR, pour Cross Data Center Replication, permet de répliquer pratiquement en temps réelle les données en provenance d'un bucket Couchbase choisi vers un index Elasticsearch. Cet index est par ailleurs automatiquement construit par une reconnaissance de la structure des documents des buckets.

Replications

REMOTE CLUSTERS						Create Cluster Reference
Name	IP/hostname					
ElasticSearch	37.187.58.70:9091					Edit Delete

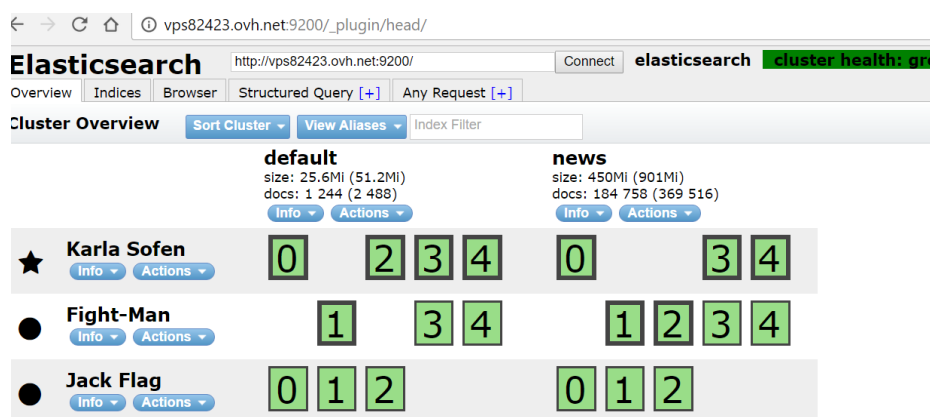
ONGOING REPLICATIONS						Create Replication
Bucket	Protocol	From	To	Status	When	
news	Version 1	this cluster	bucket "news" on cluster "ElasticSearch"	Replicating II	on change	Edit Delete

Cette connexion nécessite le plugin couchbase-transport-elasticsearch.



Dans le cadre d'un moteur de recherche sur les articles, c'est en premier lieu à Elasticsearch que les requêtes par mots clés vont s'adresser. Elasticsearch va renvoyer les ID des documents pertinents classés d'après un score de pertinence (selon le TF*IDF par défaut).

Il est possible de voir du côté ElasticSearch le comptage de documents indexés par un utilitaire web tel que _head à l'adresse suivante : http://vps82423.ovh.net:9200/_plugin/head/



Ci-dessus nous voyons l'ensemble des documents du bucket news, plus certains ajoutés par ElasticSearch, dans l'index news (index selon la terminologie ElasticSearch).

Elasticsearch peut être interrogé en queries REST. Les paramètres de la requête sont structurés en json. Les possibilités de paramétrage sont immenses. Le score de pertinence retourné va permettre de juger de la proximité entre les documents renvoyés et la requête.

Parmi les options notons :

- la possibilité de définir un écart en mots entre 2 termes de la requete.(slop).
- La tolérance à l'orthographe avec le paramètre fuzziness basé sur la distance de Levenshtein.
- Possibilité d'utiliser les opérateurs booléens
- Intégration de fonctions qui vont modifier le score selon nos besoins

Exemple de query avec le paramètres slop ou fuzziness

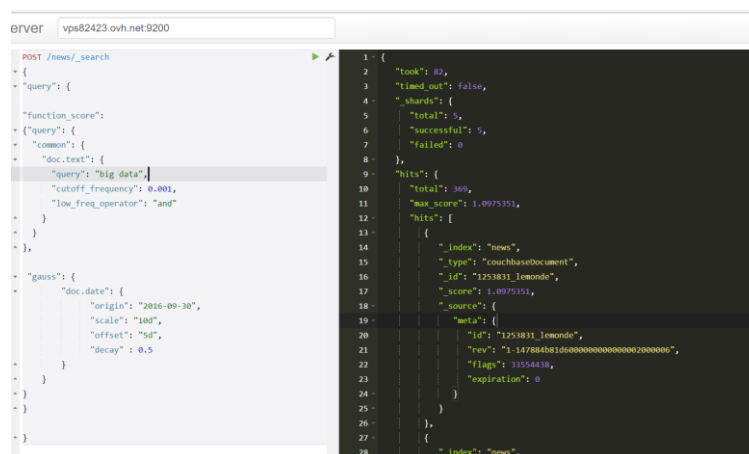
```
Server vps82423.ovh.net:9200

1 POST /news/_search
2 {
3   "query": {
4     "match": {
5       "doc.text": {
6         "type": "phrase",
7         "query": "économie espagnole",
8         "slop": 15,
9         "operator": "AND",
10        // "cutoff_frequency": 0.001,
11        // "zero_terms_query": "all",
12        "fuzziness": 2
13      }
14    }
15  }
16 }
17 }
18 }
```

```
1 {
2   "took": 64,
3   "timed_out": false,
4   "_shards": {
5     "total": 5,
6     "successful": 5,
7     "failed": 0
8   },
9   "hits": {
10    "total": 5,
11    "max_score": 0.3213633,
12    "hits": [
13      {
14        "_index": "news",
15        "type": "couchbaseDoc",
16        "_id": "877234_lemonde",
17        "score": 0.3213633,
18        "_source": {
19          "meta": {
```

Certaines options (function_score) vont permettre de combiner les différents paramètres que l'on privilégie. Par exemple le score pertinence par défaut TF*IDF peut être pondéré par l'actualité (recency) du document. Dans un moteur de recherche de news la pertinence devient relative si l'article est ancien. Dans notre moteur de recherche nous avons donc mis les paramètres qui nous paraissaient les mieux appropriés au fil des tests.

Query actuelle du moteur de recherche, ici « big data » et « 2016-09-30 » sont des variables dans le script



```

POST /news/_search
{
  "query": {
    "function_score": {
      "query": {
        "common": {
          "doc.text": {
            "query": "big data",
            "cutoff_frequency": 0.001,
            "low_freq_operator": "and"
          }
        }
      },
      "gauss": {
        "doc.date": {
          "origin": "2016-09-30",
          "scale": "10d",
          "offset": "5d",
          "decay": 0.5
        }
      }
    }
  }
}

```

```

1: {
2:   "took": 82,
3:   "timed_out": false,
4:   "_shards": {
5:     "total": 5,
6:     "successful": 5,
7:     "failed": 0
8:   },
9:   "hits": {
10:    "total": 369,
11:    "max_score": 1.0975351,
12:    "hits": [
13:      {
14:        "_index": "news",
15:        "_type": "couchbaseDocument",
16:        "_id": "1251831_lemonde",
17:        "_score": 1.0975351,
18:        "_source": {
19:          "meta": {
20:            "id": "1251831_lemonde",
21:            "rev": "1-1478845b1d600000000000000000000000",
22:            "flags": 33554438,
23:            "expiration": 0
24:          },
25:          ...
26:        },
27:        "_index": "news",

```

La query ci-dessus vise à trouver un équilibre entre la date du jour, la plus récente et la pertinence.

Elasticsearch nous renvoie les document ID. Un script php va les envoyer comme paramètres pour interroger la base Couchbase. En retour les données renvoyées vont être affichées à l'intérieur d'une page html.

Pour cette dernière étape d'affichage nous avons utilisés les bibliothèques bootstrap et jquery afin d'insérer rapidement et proprement nos données brutes dans un tableau html.

Le résultat est visible à la page suivante : <http://vps82423.ovh.net/lemonde/home.html>

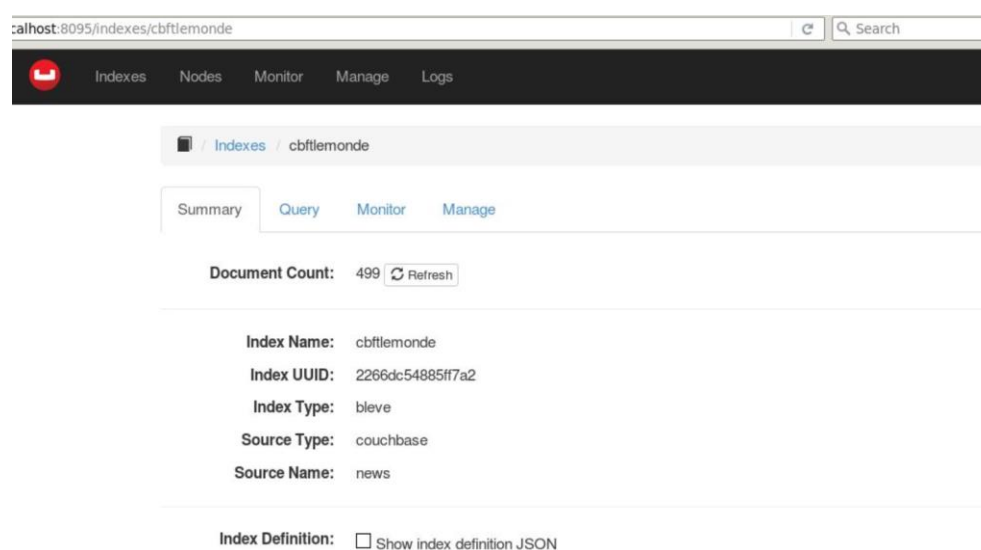
5.2 Couchbase et bleve

Intégré désormais à la dernière version commerciale de Couchbase, le moteur de recherche bleve vise les mêmes fonctionnalités que Elasticsearch. Toutefois cet outil n'a encore ni la puissance, ni la rapidité, ni les options d'Elasticsearch.

Ainsi l'indexation réalisée de la base de Couchbase a pris beaucoup plus de temps qu'avait pris la réplication de Couchbase par XDCR. Après 10mn seuls 500 documents avaient été indexés. Il faudra plusieurs heures pour indexer tous les articles.

Par ailleurs le volume occupé sur le disque par les index atteint plusieurs Giga ce qui est bien supérieur à Elasticsearch.

Interface accessible ici : <http://vps82423.ovh.net:8095/indexes/cbftlemonde>



Cependant la facilité d'installation est indéniable car les index sont créés rapidement depuis un formulaire dans une interface web.

Un moteur de recherche est également proposé depuis cette interface.

5.3 Couchbase et Spark : une combinaison prometteuse.

Parmi les connecteurs activement développés par les ingénieurs de Couchbase et régulièrement mise à jour, outre Elasticsearch il existe celui avec Spark qui est particulièrement intéressant. Spark représente un intérêt pour sa puissance de traitement des données en mémoire RAM. Il peut aider Couchbase à délivrer encore plus rapidement des données traitées. Par exemple dans le cadre d'applications mobiles. Les données stockées dans les RDD peuvent être délivrées immédiatement.

Par ailleurs Spark nous semble très intéressant et complémentaire avec Couchbase pour l'analyse de données. Spark permet en effet d'appliquer les algorithmes les plus puissants pour l'analyse textuelle tel que le LSA (Latent Semantic Analysis), LDA (Latent Dirichlet Allocator), ESA Explicit Semantic Analysis. Ces outils ouvrent des possibilités immenses de classifications des données et même pour l'indexation.

Voici des pistes à explorer en vue de réaliser un moteur de recherche sémantique qui irait donc bien au delà du niveau pertinence proposé actuellement par notre moteur de recherche sur les articles du Journal « Le Monde ». Même si Elasticsearch contribue déjà à augmenter grandement la qualité des résultats.

Référence bibliographiques :

Developing with Couchbase Server. MC Brown. *O'Reilly* 2013

Getting Started with Couchbase Server. MC Brown *O'Reilly* 2011

Learning Couchbase. Henry Postangbam. *Packt Publishing* 2015

Articles :

NoSQL showdown: MongoDB vs. Couchbase. Rick Grehan. [Infoworld](#). 2013

Accelerating Data-Driven Applications With Couchbase and Spark. Couchbase. [White Paper Couchbase](#). Juin 2016. (Disponible sur demande)